SPESIFIKASI TUGAS BESAR KOMPUTER GRAFIK 2D

Topik: *Interactive 2D Emergency Floorplan Simulator, Primitive Graphics Engine & Matrix-Driven Animation*

Platform/Engine: Godot Engine 4.x (C#)

Pendekatan: Object-Oriented Programming (OOP) & SOLID Principles

---

1. Deskripsi Umum

Mahasiswa diminta membangun aplikasi simulasi denah 2D interaktif berbasis C# di Godot Engine. Aplikasi ini berfungsi untuk merender denah bangunan fisik dari **dataset terverifikasi**, memvisualisasikan zona bahaya (api/asap), serta mensimulasikan pergerakan agen evakuasi dan efek visual menggunakan **animasi transformasi murni**.

Seluruh algoritma primitif grafik 2D, transformasi matriks, dan sistem animasi wajib diimplementasikan secara **murni (*from scratch*)** pada kanvas/custom node (`_Draw()`) tanpa mengandalkan pustaka gambar atau pustaka animasi bawaan Godot yang *black-box* (seperti `AnimationPlayer` atau `Tween`).

---

2. Dataset Denah 2D (Wajib Memilih Salah Satu)

Mahasiswa **wajib** mengimpor data denah dari salah satu dari 7 dataset *Open Source* terverifikasi berikut:

1. **CVC-FP (Database for Structural Floor Plan Analysis)**: Dataset denah arsitektural berbasis elemen struktural (dinding, pintu, jendela).
2. **MLStructFP (Machine Learning for Structural Floor Plans)**: Dataset berbasis marcom-vector/JSON yang merinci koordinat spasial dinding dan ruangan.
3. **FloorPlanCAD (Voxel51 / Hugging Face)**: Dataset CAD denah 2D berskala besar berbasis vektor primitif (*lines, polylines, arcs*).
4. **HouseExpo (TeaganLi / GitHub)**: Dataset denah 2D berbasis ruang dan *polygon/grid* yang umum digunakan pada riset simulasi evakuasi dan NavMesh.
5. **ResBIM (RogerLiang0725 / GitHub)**: Dataset ekstraksi denah arsitektural berbasis koordinat spasial BIM 2D.
6. **BIM-to-Floorplan Dataset (MDPI Data 2023)**: Dataset pasangan grafik denah 2D hasil ekstraksi dari model 3D Building Information Modeling (BIM).

7. **ResPlan (Kaggle Dataset)**: Dataset denah rumah 2D terstruktur untuk segmentasi spasial dan jalur evakuasi.

Ketentuan Parsing: Mahasiswa wajib membuat *Dataset Parser Engine* (C#) untuk membaca struktur file dataset pilihan (JSON/XML/CSV/DXF-JSON) dan menerjemahkannya menjadi himpunan koordinat $2D$.

🛠️ 3. Persyaratan Teknis & Algoritma Grafik (Wajib From Scratch)

Seluruh rendering objek wajib ditulis menggunakan logika matematis grafik murni:

A. Rendering Dinding & Sekat (Primitive Line Algorithms)

- **DDA (Digital Differential Analyzer)** & **Bresenham's Line Algorithm**.
- Wajib membuat saklar/toggle pada UI untuk membandingkan hasil rendering garis menggunakan DDA vs Bresenham secara *real-time*.

B. Rendering Titik Api & Penyebaran Asap (Circle & Ellipse Algorithms)

- **Midpoint Circle Algorithm**: Untuk merender zona radius panas/sensor lingkaran simetris.
- **Midpoint Ellipse Algorithm**: Untuk merender jangkauan penyebaran asap terdorong angin (membentuk elips horizontal/vertikal).

C. Transformasi 2D & Kamera HUD (Homogeneous Coordinates)

- Seluruh operasi **Translasi, Rotasi, dan Scaling** wajib dikalkulasi menggunakan **Matriks Koordinat Homogen (3×3)** buatan sendiri:

$$M_{total} = T(dx, dy) \cdot R(\theta) \cdot S(sx, sy)$$

- **HUD Interaktif (No Hardcode)**:
- Fitur *Zoom-In / Zoom-Out* (Scaling Matriks).
- Fitur *Pan / Drag Camera* (Translasi Matriks).
- Fitur *Rotate Floorplan* (Rotasi Matriks terhadap titik *Pivot* pusat layar).
- Fitur *Window Resize / Minimize / Maximize*: Resolusi rendering dan koordinat viewport harus menyesuaikan secara proporsional via perkalian matriks *viewport-to-screen*, **DILARANG** meng-*hardcode* piksel statis.

🎬 4. SPEK KE-2: Sistem Animasi Berbasis Composite Transformation (Wajib)

Seluruh pergerakan dan efek visual dalam simulasi **WAJIB** dihasilkan melalui **komposisi perkalian matriks homogen secara beruntun per frame** (Δt) di dalam *loop* utama

(`_Process(double delta)`). Dilarang keras menggunakan `Tween`, `AnimationPlayer`, atau interpolasi posisi langsung tanpa matriks.

$$M_{anim}(\Delta t) = T\big(x(t), y(t)\big) \cdot R\big(\theta(t)\big) \cdot S\left(s_x(t), s_y(t)\right) \cdot T\big(-x_{pivot}, -y_{pivot}\big)$$

A. Animasi Pergerakan Agen Evakuasi (Kinematic Path Tracking)

- **Komposisi Matrix**: Perkalian matriks **Translasi** (posisi agen sepanjang jalur) dan **Rotasi** (orientasi/arah hadap agen terhadap Vektor Arah $\vec{v}$).
- **Kebutuhan**: Agen bergerak mulus dari titik asal menuju pintu keluar (*exit point*). Arah badan agen wajib otomatis berputar menyesuaikan sudut belokan lorong ($R(\theta)$) saat berpindah posisi ($T(dx, dy)$).

B. Animasi Dynamic Hazard Spreading (Penyebaran Api & Asap)

- **Penyebaran Api (Pulsating Circle Expansion)**: Zona api mengembang secara dinamis menggunakan komposit perkalian matriks **Scaling** bergelombang $S(1 + \epsilon \sin(\omega t))$ yang dipadukan dengan translasi pusat api.
- **Penyebaran Asap Terhalang Ventilasi (Elliptical Wind Drift)**: Zona asap mengembang dan berputar miring mengikuti arah angin menggunakan komposit **Scaling + Rotation + Translation** pada algoritma Midpoint Ellipse.

C. Animasi Indicator / Pulse Radar Sensor (Local Pivot Scaling & Rotation)

- Sensor alarm kebakaran pada denah memiliki animasi berputar (*radar sweep*) atau berkedip (*pulsating warning icon*).
- **Aturan Pivot Point**: Animasi rotasi/skala indikator wajib dilakukan **terhadap titik pusat lokal (local pivot)** indikator tersebut, bukan terhadap *origin* layar $(0,0)$, menggunakan matriks komposisi 3-langkah:

$$M_{pivot} = T\big(x_{pivot}, y_{pivot}\big) \cdot R(\theta) \cdot S(s, s) \cdot T\big(-x_{pivot}, -y_{pivot}\big)$$

🏗️ 5. Aturan Arsitektur Kode (Prinsip SOLID & Bebas Hardcode)

Mahasiswa wajib menerapkan prinsip **SOLID** dalam arsitektur C# di Godot:

1. **Single Responsibility Principle (SRP)**:
- Pisahkan dengan tegas antara kelas **Data Loader** (Parser JSON/CSV dataset), kelas **Graphics Engine** (Kalkulasi DDA/Bresenham/Midpoint), kelas **Matrix Animation Engine**, dan kelas **Render Node** (`Node2D` Godot).
1. **Open/Closed Principle (OCP)**:
- Gunakan `interface` (misal: `ILineRenderer`, `ITransformationMatrix`, `IAnimatable`) sehingga jenis animasi atau algoritma baru dapat ditambahkan tanpa mengubah kode utama rendering.
1. **Dependency Inversion Principle (DIP)**:

- *High-level modules* (HUD Control & Animation Controller) tidak boleh bergantung langsung pada *low-level modules* (Piksel Canvas), melainkan melalui abstraksi *Camera/Animation State*.

1. **Eksternalisasi Konfigurasi (No Hardcode)**:

- Seluruh parameter aplikasi (kecepatan animasi agen, kecepatan ekspansi api, kecepatan zoom, warna garis dinding, jalur file dataset, *default resolution*) **WAJIB** disimpan dalam file konfigurasi eksternal (`config.json` atau `config.yaml`). Dilarang keras menaruh angka konstan (*magic numbers*) di dalam kode C#.

📐 6. Struktur Project Godot (C#)

```
res://
├── Config/
│   └── app_config.json             <-- File konfigurasi (Bebas Hardcode)
├── Datasets/
│   └── selected_dataset_file.json  <-- Salah satu dari 7 Dataset
Terverifikasi
├── Scripts/
│   ├── Core/
│   │   ├── ConfigLoader.cs         <-- Membaca app_config.json
│   │   └── DatasetParser.cs        <-- Membaca dataset denah pilihan
│   ├── GraphicsEngine/
│   │   ├── Interfaces/
│   │   │   ├── ILineRenderer.cs
│   │   │   ├── IShapeRenderer.cs
│   │   │   └── IAnimatable.cs      <-- Interface untuk objek teranimasi
matriks
│   │   ├── DDALine.cs              <-- Algoritma DDA murni
│   │   ├── BresenhamLine.cs        <-- Algoritma Bresenham murni
│   │   ├── MidpointCircle.cs       <-- Midpoint Circle murni
│   │   ├── MidpointEllipse.cs      <-- Midpoint Ellipse murni
│   │   └── Matrix3x3.cs            <-- Matriks Komposisi Homogen 3x3
│   ├── Animation/
│   │   ├── CompositeAnimator.cs    <-- Engine komposit matriks animasi
(_Process)
│   │   ├── AgentPathFollower.cs    <-- Animasi translasi + rotasi agen
│   │   └── HazardExpansion.cs      <-- Animasi scaling + rotasi api/asap
│   ├── Controllers/
│   │   ├── FloorplanRenderer.cs    <-- Modul Canvas (_Draw)
│   │   └── HUDCameraController.cs  <-- Kontrol Zoom/Pan/Rotate via Matriks
└── Scenes/
    └── MainSimulator.tscn
```

---

7. Rubrik Penilaian (Total: 100%)

No	Komponen Penilaian	Bobot	Indikator Utama
1	**Dataset Integration**	**10%**	Mampu memuat dan memetakan denah dari salah satu dari 7 dataset terverifikasi (CVC-FP, MLStructFP, FloorPlanCAD, HouseExpo, ResBIM, MDPI BIM, ResPlan) secara presisi.
2	**Primitive Line Algorithms**	**15%**	Implementasi DDA & Bresenham dari nol di C# beserta fitur *toggle* pembanding keduanya.
3	**Primitive Curve Algorithms**	**15%**	Implementasi Midpoint Circle (titik api) & Midpoint Ellipse (penyebaran asap) dari nol.
4	**Matrix & Homogeneous Trans.**	**15%**	Operasi Pan, Zoom, Rotate, dan Resize Window dikontrol penuh oleh perkalian Matriks Homogen 3×3 buatan sendiri.
5	**Composite Matrix Animation (Spek 2)**	**25%**	Animasi agen, ekspansi api/asap, dan indikator radar berhasil dirender murni menggunakan perkalian komposit matriks 3×3 beruntun per-frame (Δt) tanpa `Tween/AnimationPlayer`.
6	**SOLID & Clean**	**10%**	Penerapan SRP, DIP,

No	Komponen Penilaian	Bobot	Indikator Utama
	Architecture		OCP, serta eksternalisasi parameter ke `app_config.json` (bebas *magic numbers*).
7	**UI/HUD & Responsivitas**	**10%**	Interaksi HUD mulus, fleksibel saat *maximize/minimize* jendela aplikasi, dan animasi berjalan tanpa *lag/glitch*.